



The Hamiltonian Market Maker: Regime-Aware Pricing for Per- ishable Heterogeneous Compute

Zach Kelling

Hanzo AI

2026

Abstract

Constant-product market makers price a fungible, storable pair. Compute is none of those things: a GPU-second not sold at 14:00 is gone, an H100-second and a CPU-second are not substitutes at any exchange rate, and a job with a three-second deadline values capacity differently from the same job with an hour. An $x \cdot y = k$ curve has no way to express scarcity convexity, deadline pressure, or per-resource isolation, and a market that adopts one inherits those silences.

We describe an alternative that we have implemented and run: price as position and trend as momentum in a phase space, evolved by an energy-conserving integrator under a regime-dependent anharmonic potential, with the regime itself recovered as the latent state of a hidden Markov model driven by request rate, queue depth and utilisation. Each resource kind carries its own market state and prices never cross between kinds, which is the property that makes heterogeneity representable rather than averaged away. Per-tenant preference is carried as a one-bit sign delta over a base quote table, small enough that millions of tenant tables sit in memory at once.

The result is a quote that is deterministic given a job and a market state — reproducible and testable rather than emergent — and that responds to a demand spike by changing its own dynamics rather than by sliding along a fixed curve.

Contents

1	What a curve cannot say	3
2	Phase space	3
3	Regime as a latent variable	4
4	Heterogeneity by construction	4
5	Determinism as a design requirement	5
6	Per-tenant preference at one bit	5
7	Active inference in the routing decision	5
8	Two layers, and why they compose	5
9	Availability and status	6

1 What a curve cannot say

A constant-product maker holds reserves (x, y) and quotes the price implied by $x \cdot y = k$. The design is superb for what it was built for, and each of its assumptions fails for compute.

Fungibility. The curve assumes any unit of x substitutes for any other. Accelerator-seconds do not: memory capacity, memory bandwidth, interconnect and resident state make two nominally identical devices different suppliers for a given job. Averaging them into one reserve destroys exactly the information the allocator needs.

Storability. Reserves persist. Capacity does not. An idle accelerator-second is not inventory carried to the next block; it is destroyed. Perishability means the reservation price falls as the window closes, which no static curve represents.

Deadline neutrality. The curve prices quantity, not urgency. Two requests for identical resources with different service objectives are different products with different willingness to pay, and the difference is not a quantity difference.

Scarcity convexity. Near exhaustion, the marginal value of the last unit of a specific resource rises faster than any fixed hyperbola. A curve chosen in advance either underprices the tail or overprices the middle, permanently.

2 Phase space

We take a different primitive. Let q denote demand pressure and p the shadow price of a resource kind, and treat the pair as a point in phase space evolving under a Hamiltonian

$$H(q, p) = \frac{p^2}{2m} + V_\varrho(q), \quad (1)$$

where V_ϱ is a potential whose shape depends on the market regime ϱ . The equations of motion are the usual ones, with a dissipative term so the system settles rather than ringing forever:

$$\dot{q} = \frac{\partial H}{\partial p} = \frac{p}{m}, \quad \dot{p} = -\frac{\partial H}{\partial q} - \gamma p. \quad (2)$$

Price is position; trend is momentum. That is the whole of the intuition, and it buys three things a curve does not have.

Momentum is a state variable, not an inference. A market that has been rising carries that fact in p . A curve has to rediscover trend from reserve history every time it is asked.

Integration is energy-conserving. We integrate symplectically. The practical consequence is that the maker does not drift on noise: a symplectic integrator has no secular energy error, so a long run of small random shocks does not accumulate into a phantom trend. A naive Euler step on the same dynamics gains energy monotonically and would manufacture exactly the drift we are trying to avoid.

Convexity is in the potential, where it can be changed. An anharmonic potential gives the steepening response near exhaustion that a hyperbola cannot, and because the potential is a separate object it can be re-specified per regime without touching the integrator.

3 Regime as a latent variable

The potential’s dependence on ϱ is the second half of the design, and it is where the other reading of the acronym lives: the regime is the hidden state of a hidden Markov model.

Definition 1 (Market regime). *A discrete latent state ϱ_t evolving as a Markov chain with learned transition matrix A , emitting observations o_t — request arrival rate, queue depth, accelerator utilisation. The posterior $\Pr[\varrho_t \mid o_{1:t}]$ is maintained by the forward recursion; the maximum-likelihood trajectory is available by Viterbi decoding and the transition and emission parameters are fit by Baum–Welch.*

The states we carry distinguish an exploring market with high posterior uncertainty, an exploiting market with low uncertainty, a market in crisis under extreme conditions, and a market in transition between them. The regime modulates two things the maker controls directly: the width of its quotes and its inventory bias.

This composition is the substantive claim of the paper. A market maker that detects its own regime and changes its dynamics accordingly is doing something categorically different from one that slides along a fixed curve — it is not merely repricing, it is reparameterising. A demand spike does not move the quote along a hyperbola; it changes which potential the quote is evolving under.

Remark 1. *The two readings of the acronym are not a pun we are stuck with. The hidden Markov layer is a standalone sequence model, usable for state detection, sequence prediction and anomaly detection with no market attached; the market maker is a consumer of it. Keeping them separable is what allows the regime detector to be tested against synthetic sequences with known ground truth, which is not possible for a component fused into a pricing loop.*

4 Heterogeneity by construction

Each resource kind — accelerator, processor, memory, storage, bandwidth — carries its own market state, and prices never cross between kinds. This is a property we enforce rather than an emergent outcome, and it is worth being explicit about why.

A single blended compute price is a lie of composition. It says that a shortage of memory bandwidth and a shortage of accelerator throughput are the same event, and it produces quotes that are wrong in both directions simultaneously: too high for the resource in surplus, too low for the one that is binding. Per-kind isolation means a bandwidth spike raises the bandwidth quote and leaves the processor quote alone, which is both correct and legible to a provider trying to decide what to add.

Layer	Object	Property enforced
Regime detector	hidden Markov model	latent state from observable load
Dynamics	symplectic integrator	no drift on noise
Potential	anharmonic, per regime	scarcity convexity
Modulation	service-objective tightness	perishability, deadline pressure
Isolation	per-kind market state	prices never cross kinds
Bounds	floor and ceiling	a quote is never free or unbounded

Table 1: The pricing stack and the property each layer is responsible for.

5 Determinism as a design requirement

The quoting entry point is deterministic for a given job and market state. It uses the noise-free integrator path exclusively — no random draw enters a quote.

This is not fastidiousness. A market whose quotes are irreproducible cannot be audited, cannot be regression-tested, and cannot settle a dispute about what it offered. Determinism at the quoting layer means a disputed quote can be recomputed from the recorded job and market state by anyone, which converts a disagreement about pricing into an arithmetic check. It also means the equilibrium the dynamics reach is a testable object rather than a property one observes and hopes persists.

Where randomness genuinely belongs — exploration in the adaptive layer — it sits outside the quote path, in the routing policy, where its effects are measurable and can be turned off.

6 Per-tenant preference at one bit

Tenants differ in how they trade price against latency and quality, and a maker that quotes every tenant identically is leaving surplus on the table. The straightforward implementation — a full-precision preference table per tenant — does not survive contact with scale.

We carry the tenant’s deviation as a one-bit sign delta over a shared base quote table: one bit per regime-and-resource cell, with per-layer scale factors and higher-precision bias terms retained. The compression against a single-precision table is thirty-two-fold, which is the difference between holding thousands of tenant tables in memory and holding millions.

There is a pleasing symmetry here worth naming, because it is the same idea appearing at two layers of the stack. At the model layer, personalization is a low-rank or low-bit delta over frozen shared weights. At the pricing layer, personalization is a one-bit delta over a shared quote table. In both cases the shared object is large and immovable, the per-tenant object is tiny, and the economics of the whole system follow from that ratio. The companion paper on residency develops the model-layer case.

7 Active inference in the routing decision

Given a quote, something still has to choose a venue. We score candidate actions by expected free energy, decomposed into an epistemic term measuring information gain and a pragmatic term measuring progress toward the requested outcome.

The reason to prefer this over pure expected-cost minimisation is that a market maker is always operating on an estimate of its own state, and the value of an allocation includes what it teaches. Routing a job to a provider whose performance is uncertain has option value that a cost-minimising allocator scores at zero and then never resolves — the classic failure where a system converges on a locally good supplier and never discovers a better one because it never tries. Making the epistemic term explicit puts exploration in the objective rather than bolting it on as a heuristic.

The dispatcher’s output is a quote — a price and a venue — not a model identity. That separation matters: which model answers is a question for the layer above, and conflating the two produces a router that cannot be reused when the model catalogue changes.

8 Two layers, and why they compose

A complete market has two allocation problems that are easy to confuse.

The lower problem is *which resources, at what price*. That is this paper: heterogeneous, perishable, deadline-bound capacity priced by regime-aware dynamics.

The upper problem is *which intelligence, for this request*. That is a different question with a different feature space — request content, task type, context length, quality requirement — and a different answer shape. We solve it with a learned per-request router that featurises the request, scores every candidate arm by a bilinear utility, and returns the best arm subject to hard service-objective ceilings, adapting online per tenant. Its fan-out variant selects the top several arms, dispatches them concurrently, and fuses the candidate answers with a synthesizer carrying no weights of its own, so that adding a new arm is adding a row rather than retraining a policy.

The layers compose cleanly because their interfaces are narrow. The upper layer asks for capacity meeting a specification and does not care how the price was formed; the lower layer supplies a quote and does not care which model will run. A market can adopt either without the other, which is the property that makes them separately useful and jointly more than the sum.

9 Availability and status

The pricing crate is roughly forty-seven hundred lines of Rust implementing the hidden Markov core — forward-backward, Viterbi, Baum-Welch — alongside the Hamiltonian dynamics, the compute-pricing adapter, regime detection, one-bit tenant adapters, expected-free-energy routing and persistence. It is deliberately not a model-serving engine; the serving path is a separate component and the separation is enforced at the crate boundary.

We state two limitations plainly. The parameters — friction, energy scale, potential shape per regime, the bounds — are chosen and defensible rather than fit against a large live market, because the market that would fit them is what this work is proposing to build. And the regime taxonomy is coarse; four states is a starting point, and a market with real load data should expect to refit both the state count and the transitions rather than inherit ours.

Neither limitation is load-bearing for the argument. The claim is not that our parameters are right; it is that price-as-position under a regime-dependent potential is the right *shape* for perishable heterogeneous capacity, that the shape is implementable, and that we have implemented it.